

53. Le son

Quatre voix, dont trois qu'on peut jouer.

1. Ce qu'il faut comprendre

La console a une puce sonore depuis toujours, et rien ne lui parlait. `note(voix, hauteur, duree, volume)` joue une note sur la voix **1** ou **2** — deux signaux carrés — et `bruit(duree, volume)` frappe sur la quatrième, celle du bruit : une explosion, un pas, un tir.

Les hauteurs portent des noms : `D04`, `RED4`, `MI4`... de `D02` à `SI6`, cinq octaves. Un nom se relit ; un numéro de fréquence, non. La table est calculée à la compilation et gravée dans la cartouche — 120 octets pour tout le clavier.

La **durée** se compte en 256^e de seconde, de 1 à 64. Une durée de `0` veut dire « sans fin » : la note tient jusqu'à `silence(voix)`. Le **volume** va de 0 à 15.

Deux voix, c'est une mélodie et son accompagnement. Ici la voix 2 double la voix 1 une octave plus bas — `MELODIE[pas] - 12`, puisqu'une octave fait douze demi-tons.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <bruit>` → [0.83.1. La fonction bruit\(\) — un bruit](#)
- `#include <silence>` → [52.2. La fonction silence\(\) — faire taire une voix](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **789 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	67 o	← la console, à l'allumage — la mise en route de la console
main()	243 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	144 o	← CopierTuiles — 9 lettres sur 43
Notes	120 o	← note() — la table des notes de musique
"A POUR UN BRUIT"	15 o	← texte() — un texte gravé
MELODIE_0	8 o	← note() — une table gravée

Lettres gravées : ABINOPRTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,

Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

note()	× 2	127 o sur place, dans main()	+ Notes (120 o), MELODIE_0 (8 o)
bruit()	× 1	39 o sur place, dans main()	
silence()	× 2	12 o sur place, dans main()	
texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 2	10 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>      // écrit un texte à l'écran
#include <note>        // joue une note
#include <bouton>      // lit un bouton de la manette
#include <bruit>        // joue un bruit
#include <silence>     // fait taire une voix
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — MELODIE

```
const uint8_t MELODIE[] = { D04, MI4, SOL4, D05, SOL4, MI4, RE4, SOL4 };
```

3. UNE VARIABLE — PAS

```
uint8_t pas = 0;
```

4. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(3, 5, "A POUR UN BRUIT");

    while (true) {
        image();

        attente++;

        if (attente > 24) {
            attente = 0;

            note(1, MELODIE[pas], 20, 12);
            note(2, MELODIE[pas] - 12, 20, 6);

            pas++;
            if (pas >= sizeof(MELODIE)) pas = 0;
        }

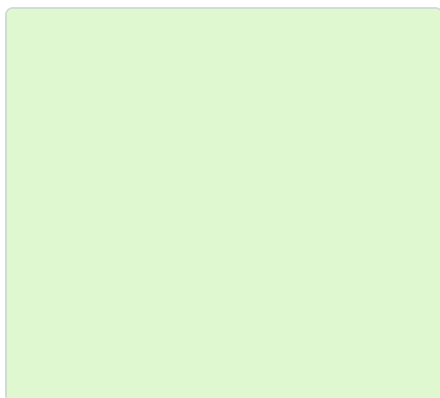
        if (bouton(A)) bruit(6, 12);
        if (bouton(START)) { silence(1); silence(2); }
    }

    return 0;
}
```

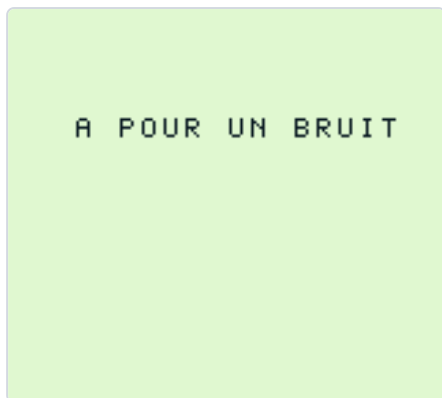
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est

la vérité sur ce programme.



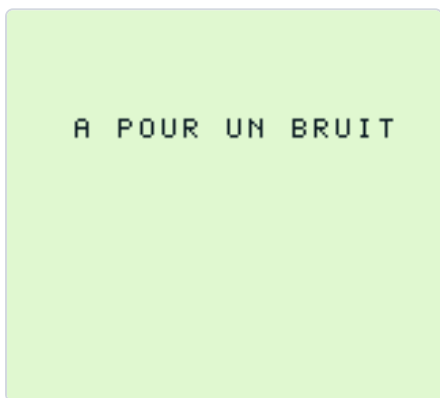
la toute première image — la console vient de démarrer — image 1



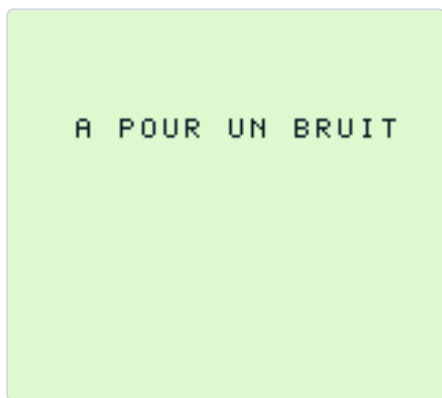
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé



START — l'écran n'a pas bougé

5. Ce qu'on doit voir

Une mélodie qui tourne en boucle ; A frappe un bruit sec.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la puce sonore est allumée
- ✓ la console produit vraiment du son (53 images sur 260)
- ✓ la voix 1 joue le DO4 et le DO5 (262, 294, 329, 392, 524 Hz)
- ✓ la voix 2 double une octave plus bas (131, 147, 165, 196, 262 Hz)

Ensuite : **53.1. Air — écrire une musique** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++